Style Transfer Implementation Summary

Here's a quick summary of how I implemented the core parts of the neural style transfer algorithm: the Gram matrix calculation and the loss functions.

1. Gram Matrix Calculation (gram_matrix Function)

I implemented the gram_matrix function to capture image style from feature maps.

First, I got the tensor dimensions (b, d, h, w). Then, I reshaped the tensor to flatten the spatial dimensions (h*w) for each channel (d). I calculated the Gram matrix using batch matrix multiplication (torch.bmm) between the reshaped features and its transpose. Finally, I normalized the Gram matrix by dividing by the total number of elements (d * h * w).

The core logic is represented by this code:

```
b, d, h, w = tensor.size()
features = tensor.view(b, d, h * w)
features_t = features.transpose(1, 2)
gram = torch.bmm(features, features_t)
num_elements = d * h * w
gram = gram / float(num_elements)
```

2. Loss Function Implementation

I defined the content, style, and total losses within the main optimization loop.

Target Features: I extracted features for the target image in each step using get_features(target, vgg).

Content Loss: I calculated this as the Mean Squared Error (MSE) between the 'conv4_2' features of the target and content images. The calculation is:

```
content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2'])**2)
```

Style Loss: I calculated this iteratively. For each specified style layer, I computed the target image's Gram matrix using my gram_matrix function (target_gram = gram_matrix(target_feature)). I retrieved the pre-calculated style Gram matrix (style_gram = style_grams[layer]). I calculated the weighted MSE between these two Gram matrices (layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram)**2)). I summed these weighted losses to get the total style_loss. (Note: Normalization is handled inside gram_matrix, so I didn't need to divide again here).

Total Loss: I combined the content and style losses using the weights alpha and beta. The calculation is: total_loss = alpha * content_loss + beta * style_loss

3. Parameter Settings

Here are the key parameters I set for the implementation:

Loss Weights: I set the content weight `alpha` to 1.0 and the style weight `beta` to 1e6. This placed a much stronger emphasis on matching the style compared to the content.

Style Layer Weights: I used a `style_weights` dictionary to assign specific weights to the different convolutional layers contributing to the style loss. This allowed me to control the influence of style features from various depths (e.g., 'conv1_1', 'conv2_1', etc.).

Layer Selection: For content representation, I used the features from the 'conv4_2' layer. For style representation, I used the layers specified as keys in the `style_weights` dictionary.

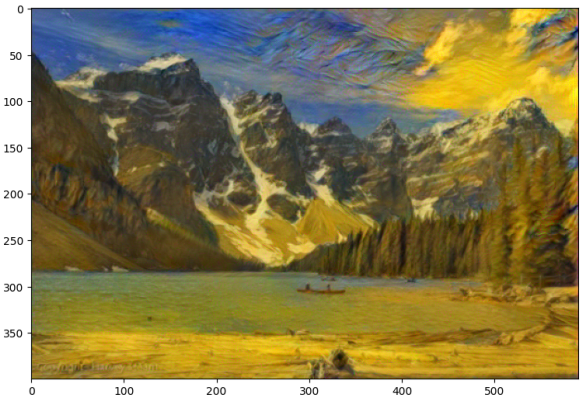
Optimizer: I chose the Adam optimizer (`optim.Adam`) to update the target image, setting the learning rate (`lr`) to 0.003.

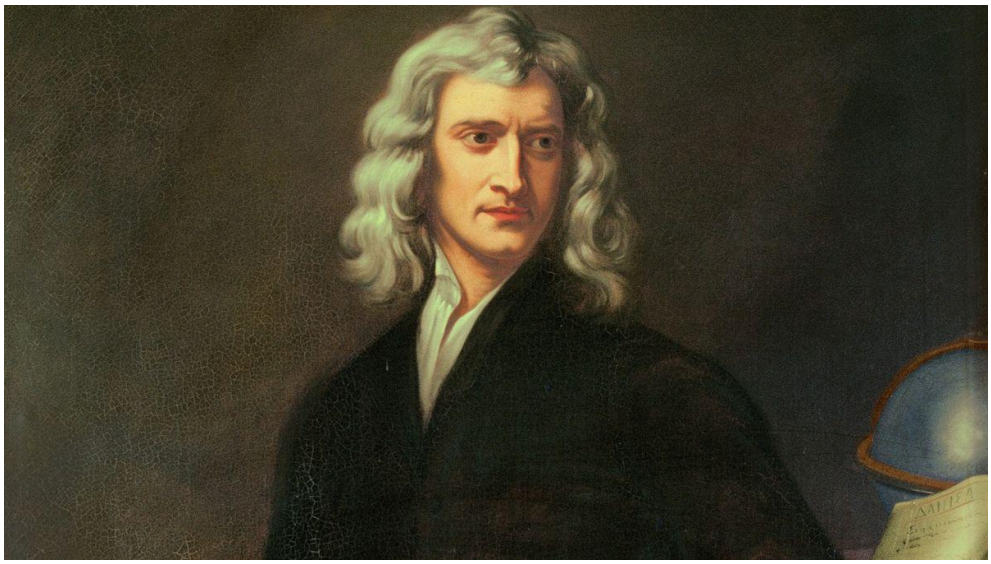
Training Duration: I ran the optimization process for 5000 steps (`steps = 5000`).

4. Visualizations

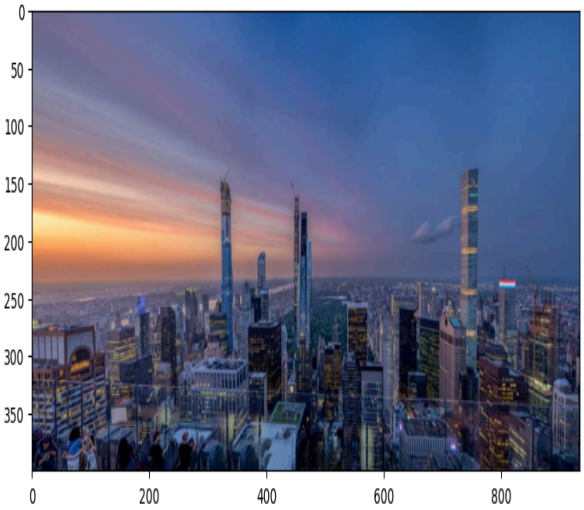
My gram_matrix function correctly computes the normalized Gram matrix. The loss functions use standard PyTorch operations and effectively leverage the gram_matrix function. Integrating normalization directly into gram_matrix simplified the style loss loop. Overall, the implementation provides the necessary calculations for the style transfer optimization.

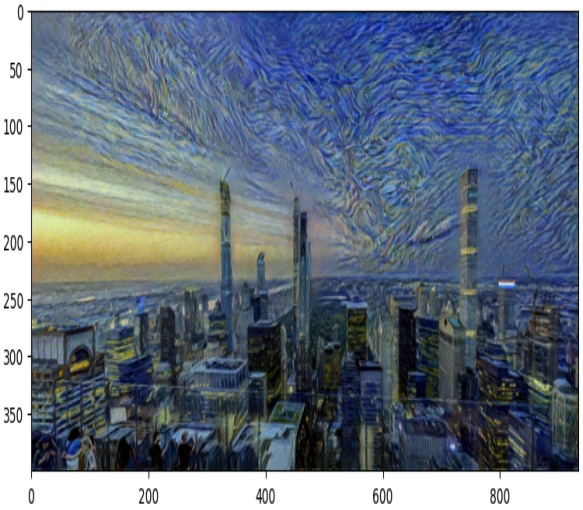
Some images after style transfer

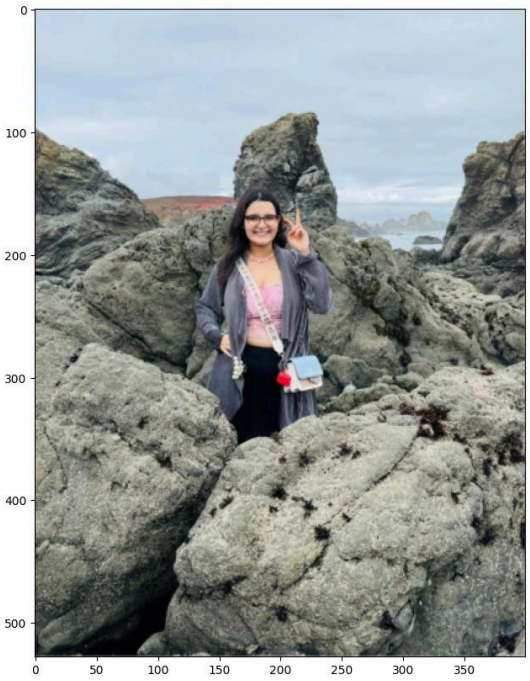


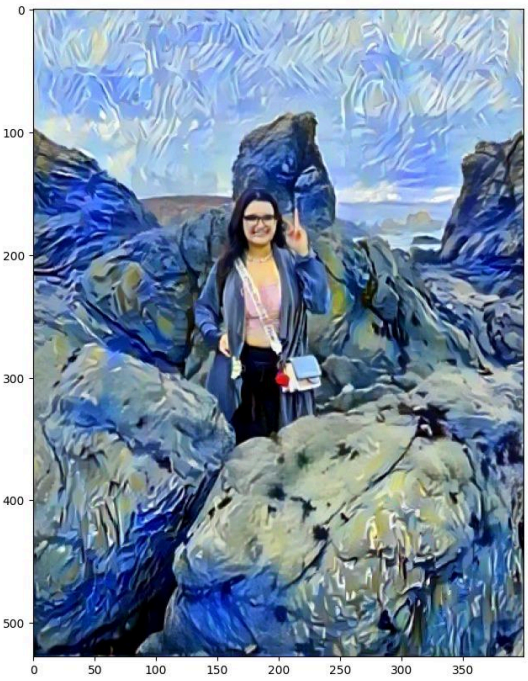



0
50
100
150
200
250
300
350
0
100
200
300
400
500
600
700


0
50
100
150
200
250
300
350
0
200
400
600
800


0
50
100
150
200
250
300
350
0
200
400
600
800


0
100
200
300
400
500
0
50
100
150
200
250
300
350


0
100
200
300
400
500
0
50
100
150
200
250
300
350